# Arboretum.hs user manual

Eugen Bronasco, Jean-Luc Falcone, Gilles Vilmart
Version: 1.0.0
https://gitlab.unige.ch/Eugen.Bronasco/arboretum.hs

October 30, 2024

## 1 Introduction

The `Arboretum.hs` package provides a suite of tools designed to facilitate computations involving algebras of graphs. Its design is guided by three core principles:

1. Readability: The implementation of structures and operations closely follows their mathematical definitions, ensuring that the code is intuitive and transparent.
2. Extensibility: The package is built to be easily extendable, allowing new features and operations to be added without significant modifications to the existing codebase.
3. Testing: Special emphasis is placed on testing. The package includes mechanisms to easily validate the correctness of implementations, reflecting a commitment to reliability.

In developing this package, we have prioritized readability and user experience over raw performance. This trade-off is deliberate, as the package is intended to serve both as a computational tool and as a platform for exploring and experimenting with algebras of graphs.

The planned features of the package include:

- B-series, which represent infinite sums of trees, and operations on them [1],
- An adjoint operator that allows users to derive coproducts from products.

Users are encouraged to explore the examples provided in the manual for a practical understanding of the package's functionality. To begin using `Arboretum.hs`, simply import the package as follows:

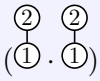
```
import Arboretum
```

Alternatively, clone the git repository and run `stack repl` in the root folder.

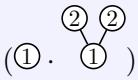
This simplicity and focus on clarity ensure that users, particularly those interested in algebraic structures, can leverage `Arboretum.hs` with ease, while also retaining the flexibility to extend its functionality as needed.

**Usage Example 1.1.** Below you can find some examples of the package usage where we define two planar forests `f1` and `f2`, graft `f1` onto `f2`, and substitute the vertices decorated by `1` in `f2` by trees of `f1`. The details can be found in Section 3.2.

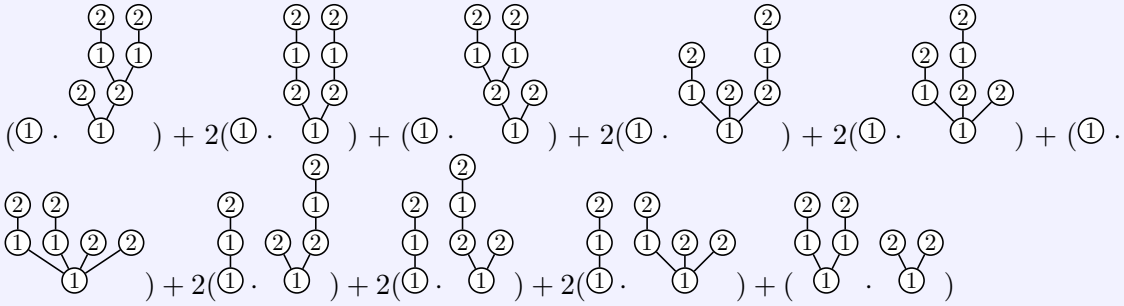
```
f1 = [
  PT 1 [PT 2 []],
  PT 1 [PT 2 []]
]
display $ vector f1
```



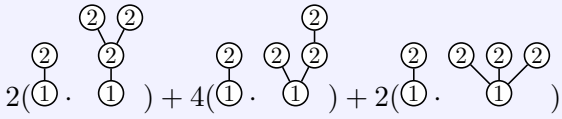
```
f2 = [
  PT 1 [],
  PT 1 [PT 2 [], PT 2 []]
]
display $ vector f2
```



```
display $ f1 `graft` f2
```



```
display $ substitute [PT 1 []] (map (:[]) f1) f2
```



## 1.1 Why Haskell?

Haskell is a strong choice for the Arboretum.hs package, aligning well with our goals of readability, extensibility, and testing. Below, we explore its key advantages, while addressing some potential disadvantages, especially in comparison to languages like Julia, Matlab, and Python.

### 1.1.1 Advantages of Haskell

1. The syntax is well-suited for writing code that mirrors mathematical definitions, particularly in domains like algebraic structures. This close alignment between code and mathematical notation enhances clarity, making the code easier to understand and reason about. Recursive structures, such as trees, are especially intuitive to implement in Haskell due to its functional paradigm.

2. Haskell enforces immutability, meaning data cannot be altered once created. This property simplifies reasoning about program behavior and ensures that functions remain free from side effects.

3. A robust, static type system enables the definition of custom types that reflect the algebraic structures in the package, allowing errors to be caught at compile time. This provides an additional layer of safety and encourages reusable, extendable code. In comparison, while languages like Python and Matlab offer flexibility, their dynamic typing can lead to runtime errors that are harder to catch.

4. Haskell's lazy evaluation model means that computations are deferred until their results are required. This is particularly advantageous for dealing with infinite structures, such as the B-series mentioned in the package's planned features. This allows for handling infinite sums efficiently without running into performance bottlenecks, a flexibility not as easily managed in languages like Julia or Matlab.

5. Haskell's emphasis on composability and modularity makes the codebase easy to extend. New structures and operations can be introduced without disrupting the existing framework, fostering scalability.

6. Haskell's support for property-based testing, through libraries like QuickCheck, allows us to define properties that algebraic structures should satisfy and automatically generate test cases to ensure correctness. This level of verification is harder to achieve in dynamically-typed languages like Python, where more reliance is placed on unit tests and manual validation.

### 1.1.2 Disadvantages and Remedies

1. While Haskell prioritizes readability and correctness, it may not always match the performance of Julia, Matlab, or Python, particularly in numerical computations. Performance bottlenecks can be mitigated by using Foreign Function Interface

(FFI) which enables integration with high-performance code from other languages, allowing Haskell to leverage the strengths of Julia, Python, or C for specific tasks.

2. Haskell's functional programming model and emphasis on immutability can be challenging for developers accustomed to more imperative languages like Python, Matlab, or Julia. These languages are often easier to learn offering familiar syntax and workflows. However, the Arboretum.hs package is written without the use of advanced Haskell techniques which reduces the complexity and makes it more approachable for new users.

In summary, while Haskell may not offer the same performance or immediate accessibility as Julia, Matlab, or Python in some areas, it excels in readability, correctness, and modularity. With the right optimizations and careful design, Haskell is a robust and extendable tool for working with algebras of graphs.

## 2 Vector spaces

As the core objective of this package is to implement and manipulate algebras of graphs, we first define the fundamental structure on which these algebras are built: vector spaces. This section provides a comprehensive explanation of vector spaces, their implementation, and the available operations for their manipulation. We assume that the vector spaces are graded, with each homogeneous subspace being finite-dimensional, as outlined in Assumptions 2.2 and 2.3.

**Definition 2.1.** A **vector space** is a set $V$ and a field of scalars $F$ with two operations: addition $+$ and scalar multiplication $\cdot$ that satisfy the following properties:

(1) $V$ is an abelian group under addition,

(2) for all $a, b \in V$ and $\lambda \in F$, we have $\lambda \cdot (a + b) = \lambda \cdot a + \lambda \cdot b$,

(3) for all $a \in V$ and $\lambda, \mu \in F$, we have $(\lambda + \mu) \cdot a = \lambda \cdot a + \mu \cdot a$ and $(\lambda\mu) \cdot a = \lambda \cdot (\mu \cdot a)$,

(4) for all $a \in V$, we have $1 \cdot a = a$.

The vector space $V$ is **graded** if it can be written as a direct sum of its subspaces

$$V = \bigoplus_{n \in \mathbb{N}} V_n.$$

**Assumption 2.2.** All vector spaces are assumed to be graded.

**Assumption 2.3.** Given a vector space $V = \bigoplus_{n \in \mathbb{N}} V_n$, all $V_n$ are finite-dimensional.

Assumptions 2.2 and 2.3 allow us to manipulate formal series as long as the manipulations act nicely with respect to the grading. Let $G$ be a set of basis elements graded

by a function $gr : G \to \mathbb{N}$ that generates the vector space $V$, then, any element $v$ of $V$ can be represented as

$$v = \sum_{n \in \mathbb{N}} \sum_{g \in gr^{-1}(n)} \langle v, g \rangle g, \tag{1}$$

where $\langle v, g \rangle$ denotes the coefficient of $g$ in $v$.

**Implementation 2.4.** We define the elements of a vector space below.

```
data ScalarProduct k b = SP
    { scalar :: k
    , basisElement :: b
    }
data Sum k b = S Integer (ScalarProduct k b) (Sum k b) | Zero
data Vector k b = V (Sum k b) (Vector k b) | Empty
```

where

1. `ScalarProduct k b` is a pair of a scalar `k` and a basis element `b`,
2. `Sum k b` is a recursively defined list (assumed finite) of scalar products with basis elements of the same grading which is stored in the first argument of `S`,
3. `Vector k b` is a recursively defined list (possibly infinite) of sums.

We note that the implementation reproduces Equation (1). However, since it is not possible to verify if a list is infinite, the implementation does not enforce the finiteness of `Sum k b`, but instead assumes it. We note that the implementation relies heavily on the Assumptions 2.2 and 2.3.

The basis, denoted as `b`, must be an instance of the `Eq` and `Graded` type classes. An instance of `Eq` ensures that we can determine whether two elements of type `b` are equal, which is a fundamental requirement for most types. To be an instance of the `Graded` type class, a type `b` must implement a function `grading :: b -> Integer`, which assigns a grading (or degree) to each element. The definition of the `Graded` type class is given below.

**Implementation 2.5.** `Graded` type class.

```
class Graded b where
    grading :: b -> Integer
```

Common types like `Integer` and `Char` have instances of the `Graded` class, where each integer or character is assigned a grading of 1. For composite structures such as lists, 2-tuples, and multisets, the grading is defined as the sum of the gradings of their elements.

The scalar field, `k`, is required to be an instance of both the `Num` and `Eq` type classes. An instance of `Num` enables arithmetic operations on elements of type `k`, and most numeric types provide this functionality. As with `Eq`, the `Num` instance is a standard requirement for types representing numerical values.

## 2.1 Constructing vectors

A vector is constructed from data types that have an instance of the `IsVector` type class, which includes all types listed in Implementation 2.4. The definiton of the `IsVector` type class is shown below.

**Implementation 2.6.** The `IsVector` type class leverages `TypeFamilies`, specifically `VectorScalar` and `VectorBasis`, which map the type `v` to its associated scalar and basis types.

```
class IsVector v where
    type VectorScalar v
    type VectorBasis v
    vector :: v -> Vector (VectorScalar v) (VectorBasis v)
```

As an example, the instance of `IsVector` for the type `ScalarProduct k b` is presented below.

**Implementation 2.7.** The instance for `ScalarProduct k b` converts a scalar product into a sum and then uses the instance for `Sum k b` to obtain the corresponding vector.

```
instance (Num k, Eq k, Eq b, Graded b)
        => IsVector (ScalarProduct k b) where
    type VectorScalar (ScalarProduct k b) = k
    type VectorBasis (ScalarProduct k b) = b
    vector = vector . (+: Zero)
```

We can construct finite vectors using the `vector` function along with the operations `*^` and `+:`. The `*^` operation is used to form scalar products, while `+:` constructs sums. The `+:` operation ensures that sums contain distinct basis elements of the same grading. To render the vector in LaTeX and PDF, we use the `display` function.

**Usage Example 2.8.** Construct a vector from a scalar product:

```
>>> v1 = vector $ 3 *^ 'a'
>>> display v1
```

$3a$

and from a sum of terms with the same grading:

```
>>> v2 = vector $ 1 *^ 'x' +: 2 *^ 'y' +: 3 *^ 'z' +: Zero
>>> display v2
```

$x + 2y + 3z$

To construct a finite vector from a finite list of scalar products with varying gradings, use

`vectorFromList :: [ScalarProduct k b] -> Vector k b.`

For an infinite vector (formal series), constructed from a potentially infinite list of scalar products with non-decreasing gradings, use

`vectorFromNonDecList : [ScalarProduct k b] -> Vector k b.`

**Usage Example 2.9.** Construct a vector from a finite list of scalar products with different gradings:

```
>>> v1 = vectorFromList
       [3 *^ ['a'], 2 *^ ['b', 'c'], 1 *^ ['c','d','e']]
>>> display v1
```

$3a + 2(b \cdot c) + (c \cdot d \cdot e)$
or from a possibly infinite list of scalar products with non-decreasing gradings:

```
>>> v2 = vectorFromNonDecList [i *^ replicate i 'a' | i <- [1..]]
>>> display $ takeV 5 v2
```

$a + 2(a \cdot a) + 3(a \cdot a \cdot a) + 4(a \cdot a \cdot a \cdot a) + 5(a \cdot a \cdot a \cdot a \cdot a)$
where `takeV n` extracts the first $n$ terms of a vector.

## 2.2 Manipulating vectors

Once constructed, vectors can be both added and scaled. Recall that a vector space forms a group under addition, so we use the group operation `<>` to add vectors and the `invert` function to negate them. To scale a vector, we apply the `scale` function, which multiplies each scalar component of the vector by a specified scalar.

**Usage Example 2.10.** Group operation on vectors:

```
>>> v1 = vector $ 3 *^ 'a' +: 2 *^ 'b' +: 1 *^ 'c' +: Zero
>>> v2 = vector $ 1 *^ 'a' +: 2 *^ 'b' +: 3 *^ 'c' +: Zero
>>> display $ v1 <> v2
```

$4a + 4b + 4c$
and negation of a vector:

```
>>> display $ invert v1
```

$-3a + -2b + -c$
combining them produces:

```
>>> display $ v1 <> (invert v2)
```

$2a + -2c$
Scaling of a vector:

```
>>> display $ scale 2 v1
```

$6a + 4b + 2c$

To extend a map `f` defined on the basis to the entire vector space, we use the `linear` function, or its variants `linearGraded` and `linearNonDec`. It is important to note that for each basis element `b`, the value `f b` must belong to a type that is an instance of the `IsVector` type class.

**Usage Example 2.11.** Linear map on a vector:

```
>>> f x = case x of
      'x' -> 1 *^ 'a';
      'y' -> 2 *^ 'b';
      'z' -> 3 *^ 'c'
>>> v = vector $ 1 *^ 'x' +: 2 *^ 'y' +: 3 *^ 'z' +: Zero
>>> display $ linear f v
```

$a + 4b + 9c$

To exclude specific scalar products from a vector, we provide three operations analogous to those on lists: `filterV`, `takeV`, and `takeWhileV`. Among these, `takeV` is particularly useful when dealing with infinite vectors (formal series), as demonstrated in Usage Example 2.9.

## 2.3 Commutative and non-commutative algebras

When the basis of a vector space forms a monoid, we can define a product on the vector space by extending the monoid's product to the entire space. This product is bilinear,

and the resulting vector space, $V$, is referred to as an algebra. Whether the algebra is commutative depends on the commutativity of the product on the basis.

A free associative algebra is a vector space whose basis consists of words over an alphabet, with the product defined as word concatenation. This product may be commutative or non-commutative. In the implementation, the free associative algebra is represented as the vector space `Vector k [b]`, where `[b]` denotes a list of elements of type `b`. Examples of vectors of type `Vector k [b]` can be found in Usage Example 2.9. For commutative algebras, `[b]` is replaced by `MultiSet b`.

To simplify the implementation of morphisms from free algebras, we provide the `morphism` function, along with its `Graded` and `NonDec` variants. This function extends a map `f`, which sends elements of type `b` to a monoid, to a full algebra morphism. For example:

**Usage Example 2.12.** We use morphism below:

```
>>> f x = case x of
        'x' -> 1 *^ ['a'];
        'y' -> 2 *^ ['b'];
        'z' -> 3 *^ ['c'];
        'w' -> 4 *^ ['d']
>>> v = vector
        $ 1 *^ ['x','y'] +: 2 *^ ['y','z'] +: 3 *^ ['z','w'] +: Zero
>>> display $ morphism f v
```

$2(a \cdot b) + 12(b \cdot c) + 36(c \cdot d)$

To handle more complex product constructions, the package provides the `bilinear` function. Here's an example:

**Usage Example 2.13.** We use bilinear below:

```
>>> f x y = 1 *^ [x, y]
>>> v1 = vector
        $ 1 *^ 'x' +: 2 *^ 'y' +: 3 *^ 'z' +: Zero
>>> v2 = vector
        $ 10 *^ 'x' +: 20 *^ 'y' +: 30 *^ 'z' +: Zero
>>> display $ bilinear f v1 v2
```

$10(x \cdot x) + 20(y \cdot x) + 20(x \cdot y) + 30(z \cdot x) + 40(y \cdot y) + 30(x \cdot z) + 60(z \cdot y) + 60(y \cdot z) + 90(z \cdot z)$

Additionally, when working with algebras of the form `Vector k m`, where `m` is a monoid, the vector space `Vector k m` has a `Num` instance. This allows us to use familiar arithmetic notation for addition and multiplication. For example:

**Usage Example 2.14.** We use the $+$ and $*$ operations below:

```
>>> v1 = vector
        $ 1 *^ ['x'] +: 2 *^ ['y'] +: 3 *^ ['z'] +: Zero
>>> v2 = vector
        $ 1 *^ ['a'] +: 2 *^ ['b'] +: 3 *^ ['c'] +: Zero
>>> display $ v1 + v2
```

$x + 2y + 3z + a + 2b + 3c$

```
>>> display $ v1 * v2
```

$(x \cdot a) + 2(y \cdot a) + 2(x \cdot b) + 3(z \cdot a) + 4(y \cdot b) + 3(x \cdot c) + 6(z \cdot b) + 6(y \cdot c) + 9(z \cdot c)$

The deshuffle coproduct takes a word and sums over all possible ways to split it into two parts—left and right—by placing each variable in either part. We have implemented the `deshuffleCoproduct` function, which is particularly useful for the recursive implementation of products in combinatorial contexts. For example:

**Example 2.15.** An example of the deshuffle coproduct is:

$$\Delta(xyz) = xyz \otimes \mathbf{1} + yz \otimes x + xz \otimes y + xy \otimes z + z \otimes xy + y \otimes xz + x \otimes yz + \mathbf{1} \otimes xyz.$$

**Definition 2.16.** Let $V$ be a vector space and $\mathcal{T}(V)$ be the corresponding tensor algebra. **Deshuffle coproduct** is a linear map $\Delta : \mathcal{T}(V) \to \mathcal{T}(V) \otimes \mathcal{T}(V)$ defined as follows:

$$\begin{aligned} \Delta(v) &= \mathbf{1} \otimes v + v \otimes \mathbf{1}, \quad \text{for } v \in V, \\ \Delta(a \cdot b) &= \Delta(a) \cdot^{\otimes} \Delta(b), \quad \text{for } a, b \in \mathcal{T}(V), \end{aligned}$$

where $(a \otimes b) \cdot^{\otimes} (c \otimes d) = (a \cdot c) \otimes (b \cdot d)$.

**Implementation 2.17.** Implementation of the deshuffle coproduct:

```
deshuffleCoproduct :: [b] -> Vector ([b], [b])
deshuffleCoproduct =
    product
        . map (\b -> vector ([], [b]) + vector ([b], []))
```

The function `map :: (a -> b) -> [a] -> [b]` applies a function to every element of a list, and `product :: [a] -> a` takes a product of all elements in a list.

A comprehensive list of available functions can be found in the documentation of the `Symbolics.hs` module at `[URL]`.

### 2.4 Displaying vectors

Efficient vector manipulation benefits greatly from having a simple and intuitive way to display them. The `display` function provides this by rendering vectors in a human-readable format, as demonstrated in all the previous usage examples. It generates a TeX representation of the vector and compiles it into a PDF named `output.pdf`, and if you are using Linux with the Zathura PDF viewer installed, the function will attempt to open the PDF using Zathura. The corresponding TeX code can be retrieved from the `output.tex` file.

This function is built using the `Texifiable` type class, which offers a TeX representation of a value. A vector of type `Vector k b` has an instance of the `Texifiable` type class as long as its basis `b` has such an instance. Below is a simplified definition of the `Texifiable` type class:

**Implementation 2.18.** Simplified definition of the `Texifiable` type class:

```
class Texifiable b where
    texify :: b -> String

    --- [...]
```

A comprehensive list of available functions can be found in the documentation of the `Output.hs` module at `[URL]`.

## 3 Algebras of graphs

We have now arrived at the core of the manual, where we present several examples of implementing algebras of graphs, showcasing and explaining the majority of the package's functionality. When constructing an algebra of graphs, two primary approaches can be adopted:

1. Defining a general graph and subsequently restricting the type of graphs we accept by imposing additional structure (as demonstrated in Section 3.1),
2. Building the specific graphs of interest directly from the ground up (as demonstrated in Section 3.2).

Each approach comes with its own set of advantages and drawbacks. The first approach is more flexible and can accommodate a broader variety of graphs, but it may introduce additional complexity and reduce efficiency. The second, more specialized approach, is often simpler and more efficient to implement, although it can require more upfront work to define the specific graphs.

In our examples, we focus on the grafting product and the associated algebra of forests and aromatic forests. Our motivation stems from numerical analysis, where the grafting product and forest formalism play an essential role.

The forest formalism is employed in numerical analysis to represent differential operators, which arise naturally when performing a Taylor expansion of the exact or approximate solution of an ordinary differential equation (ODE). In this context, vertices correspond to the vector field of the ODE, while edges represent directional derivatives.

Using this formalism, the application and composition of differential operators can be represented by operations on forests, specifically the *grafting* and *Grossman-Larson* products.

Additionally, techniques such as *backward error analysis* and *modified equations* replace the vector field of an ODE with a formal series of elementary differentials. In the forest formalism, this corresponds to substituting the vertices of a forest with trees, as described in more detail in [1].

Finally, we note that the forest formalism is extended in geometric numerical integration to aromatic forests, and in stochastic differential equations (SDEs) to exotic aromatic forests. Both aromatic and exotic aromatic forests are technically not forests but rather more general types of graphs, introducing additional challenges in their implementation.

## 3.1 Grafting of graphs

In this section, we discuss the definition and implementation of graphs, along with the grafting operation. We provide an example of implementing an algebra of graphs, starting with a general graph and imposing only the minimal required structure to define the product of interest, which is the grafting product in our case.

**Definition 3.1.** A **graph** $g$ is a set of vertices $V(g)$ together with a set of directed edges $E(g)$. An edge $e$ is defined by its source $s(e) \in V(g)$ and target $t(e) \in V(g)$. A **rooted** graph has a marked vertex called a root. The set of rooted graphs is denoted by $G_R$.

Too much code?

We implement the Definition 3.1 below:

**Implementation 3.2.** A graph is any type `g` that is an instance of the `Graph` type class. The `Graph` type class requires the type of the edges `Edge g` to be an instance of the `GraphEdge` type class, which defines the source, target, and construction of edges. The `Graph` type class also requires the type of the vertices `Vertex g` to be the `EndPoint` of the edges. The `Graph` type class provides functions for constructing graphs, adding edges, and adding graphs together.

```
class GraphEdge e where
    type EndPoint e

    source :: e -> EndPoint e
    target :: e -> EndPoint e
    edge :: a -> EndPoint e -> EndPoint e -> e
```

```haskell
class
    ( GraphEdge (Edge g)
    , Vertex g ~ EndPoint (Edge g)
    ) => Graph g where
    type Edge g

    type Vertex g
    type Vertex g = EndPoint (Edge g)

    singleton :: Vertex g -> g
    edges :: g -> MultiSet (Edge g)
    vertices :: g -> MultiSet (Vertex g)
    addGraph :: (Graph g0, Edge g ~ Edge g0) => g0 -> g -> g
    addEdge :: Edge g -> g -> g

class (Graph g) => RootedGraph g where
    root :: g -> Vertex g
```

A naive implementation of a graph is as follows:

**Implementation 3.3.** A simple implementation of a graph:

```haskell
instance GraphEdge (a, a) where
    type EndPoint (a, a) = a
    edge _ = (,)
    source (x, _) = x
    target (_, y) = y

data IntegerGraph
    = IG (MS.MultiSet Integer) (MS.MultiSet (Integer, Integer))

instance Graph IntegerGraph where
    type Edge IntegerGraph = (Integer, Integer)

    singleton v = IG (singleton v) empty
    edges (IG _ es) = es
    vertices (IG vs _) = vs
    addGraph g (IG vs es) =
        IG (vertices g `union` vs) (edges g `union` es)
    addEdge e (IG vs es) =
        IG vs (e `insert` es)

data Rooted g = R (Vertex g) g
```

```
instance (Graph g) => RootedGraph (Rooted g) where
    root (R r _) = r
```

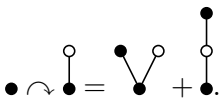
We are now ready to define and implement a generalization of the grafting product. Let $\mathcal{G}$ and $\mathcal{G}_R$ denote the vector spaces spanned by $G$ and $G_R$, respectively.

**Definition 3.4.** The **grafting product** $\curvearrowright : \mathcal{G}_R \otimes \mathcal{G} \to \mathcal{G}$ is a bilinear map defined by taking the sum over all the ways to connect the root of the left operand to a vertex of the right operand. That is,

$$g_r \curvearrowright g = \sum_{v \in V(g)} g_r \curvearrowright_v g,$$

where the operation $\curvearrowright_v$ connects the root of $g_r$ to the vertex $v$ of $g$.

For example,



The implementation of the grafting product that closely resembles the Definition **??** is presented below.

**Implementation 3.5.** Implementation of the grafting product:

```
graftGraph
    :: ( Eq a2, Graded a2, Graph a2
       , RootedGraph a1, Edge a1 ~ Edge a2
       )
    => a1 -> a2 -> Vector Integer a2
graftGraph rg1 g2 =
    vectorFromNonDecList $
        map ((1 *^) . graftGraphTo rg1 g2) $
            toList $
                vertices g2

graftGraphTo
    :: ( RootedGraph a1
       , Graph a2, Edge a1 ~ Edge a2
       )
    => a1 -> a2 -> Vertex a2 -> a2
graftGraphTo rg1 g2 v = addGraph rg1 $ addEdge new_edge g2
  where
    new_edge = edge () (root rg1) v
```

We use the `bilinear` function discussed in the previous section to apply the grafting product to vectors of graphs.

**Usage Example 3.6.** Grafting of graphs where we define the graphs using a helper function `integerGraph` that converts lists to multisets:

```
>>> g1 = rooted (integerGraph [1] []) 1
>>> g2 = rooted (integerGraph [2] []) 2
>>> g3 = integerGraph [3,4] [(4,3)]
>>> g4 = integerGraph [5] [(5,5)]
>>> bilinear graftGraph
      (vector $ 1*^g1 +: 2*^g2 +: Zero)
      (vectorFromList [3*^g3,4*^g4])
(4 *^ IntegerGraph(V=[1,5], E=[(1,5),(5,5)])
      + 8 *^ IntegerGraph(V=[2,5], E=[(2,5),(5,5)]))_2
      + (3 *^ IntegerGraph(V=[1,3,4], E=[(1,3),(4,3)])
      + 3 *^ IntegerGraph(V=[1,3,4], E=[(1,4),(4,3)])
      + 6 *^ IntegerGraph(V=[2,3,4], E=[(2,3),(4,3)])
      + 6 *^ IntegerGraph(V=[2,3,4], E=[(2,4),(4,3)]))_3
```

We note that `g1` and `g2` are rooted graphs while `g3` and `g4` are graphs.

We note that Usage Example 3.6 does not use the TeX representation of the graphs to display the result of the product. This is because the `IntegerGraph` type lacks a `Texifiable` instance (see Section 2.4), as there is no clear way to represent a general graph in TeX.

## 3.2 Grafting and substituting of decorated forests

In this section, we take an alternative approach by constructing the graphs we are interested in from the ground up. This approach allows us to leverage their properties to implement efficient formulas for operations and ensures that the structure we aim to preserve is maintained through these operations—a guarantee provided by Haskell's strong and flexible type system. However, this approach requires a more detailed understanding of the graph's structure compared to the one discussed in Section 3.1.

We will explore the definition and implementation of decorated forests, along with grafting, Grossman-Larson, and substitution operations. Later, we extend the formalism to include aromatic forests, incorporating grafting and substitution operations as well.

### 3.2.1 Decorated forests

Decorated forests are graphs in which every connected component is a decorated tree. A tree is a connected graph with a unique vertex called *the root of the tree* and a unique path from every vertex to the root. A tree $\tau$ is decorated if, given a set $D$, there exists

a map $\alpha : V(\tau) \to D$ which assigns a possibly non-unique decorator to each vertex of $\tau$. All trees and forests considered in this section are decorated, so we will often omit mentioning this for simplicity. Below you can find an equivalent definition of decorated trees which is more suitable for implementation.

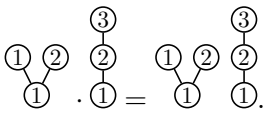
**Definition 3.7.** Given a set of decorations $D$, a **tree** decorated by $D$ is a tuple $(r, \pi)$ of a root $r \in D$ and a collection $\pi$ of trees decorated by $D$ called **children** of $r$.

Let the set of trees be denoted by $T$. We note that the collections of trees $\pi$ are also called **forests** and their set is denoted by $F$.

**Implementation 3.8.** We implement the class type `IsTree` for trees:

```
class IsDecorated a where
    type Decoration a

class (IsDecorated t) => IsTree t where
    root :: t -> Decoration t
    children :: t -> [t]
    buildTree :: Decoration t -> [t] -> t
```

The vector space of forests $\mathcal{F}$ forms an algebra with concatenation product. For example,



### 3.2.2 Planar and non-planar forests

A forest is *planar* if the order of its trees and the branches within the trees matters. In a *non-planar* forest, the order of the trees and their branches does not matter. Both classes of forests play important roles, but from an implementation perspective, planar forests are simpler to handle since they rely on list structures, whereas non-planar forests rely on multiset structures. Working with lists leads to cleaner code. Moreover, mathematically, non-planar trees can be viewed as a subspace or a quotient space of planar trees. To enable seamless transitions between planar and non-planar spaces, the `Planarable` type class is implemented.

**Implementation 3.9.** We implement the `Planarable` type class:

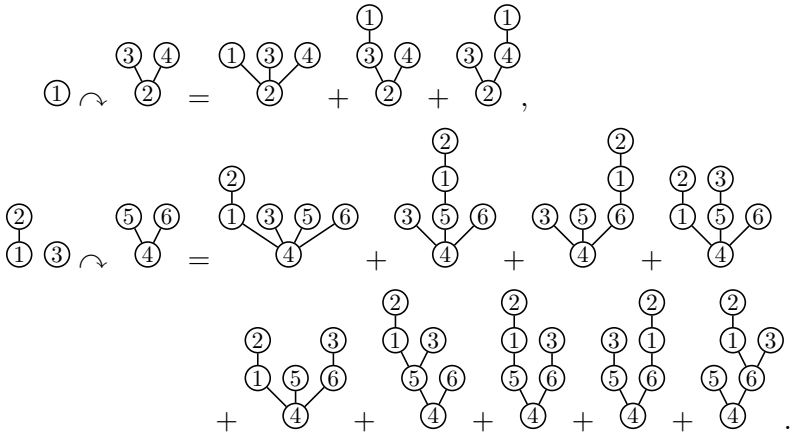
```
class Planarable t where
    type Planar t

    planar :: t -> Planar t
```

```
nonplanar :: Planar t -> t
```

### 3.2.3 Grafting

Recall the deshuffle coproduct introduced in Definition 2.16. We use Sweedler notation to denote the deshuffle coproduct as $\Delta(\pi) = \sum_{(\pi)} \pi^{(1)} \otimes \pi^{(2)}$ and consider the operation of grafting on decorated forests. The forest $\pi_1$ is grafted onto the forest $\pi_2$ by taking the sum over all ways to attach the roots of the forest $\pi_1$ to the vertices of $\pi_2$.

**Example 3.10.** Some examples of grafting are:



**Definition 3.11.** **Grafting** of two empty forests gives an empty forest:

$$\mathbf{1} \curvearrowright \mathbf{1} := \mathbf{1},$$

forest on an empty forest is 0:

$$\pi \curvearrowright \mathbf{1} := 0, \quad \text{for } \pi \neq \mathbf{1},$$

forest on a tree is defined using Grossman-Larson product:

$$\pi \curvearrowright \tau := (r, \pi_1 \diamond \pi_2), \quad \text{for } \tau = (r, \pi_2) \in T,$$

forest on a non-empty forest is defined using deshuffle coproduct:

$$\pi_1 \curvearrowright (\tau \cdot \pi_2) := \sum_{(\pi_1)} (\pi_1^{(1)} \curvearrowright \tau) \cdot (\pi_1^{(2)} \curvearrowright \pi_2), \quad \text{for } \pi_1, \pi_2 \in F.$$

with $(r, -)$ being linear and $\diamond$ denoting the **Grossman-Larson** product defined as

$$\pi_1 \diamond \pi_2 := \sum_{(\pi_1)} \pi_1^{(1)} \cdot (\pi_1^{(2)} \curvearrowright \pi_2).$$

We note that Definition 3.11 implies $\mathbf{1} \curvearrowright \pi = \pi$.

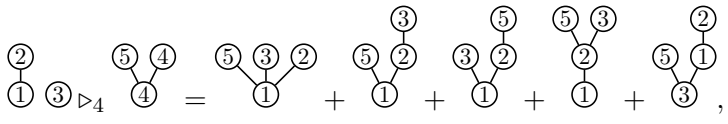
**Implementation 3.12.** To facilitate future extension of the algebra to include aromatic forests, we define the `Graftable` type class for graphs that support grafting operations:

```
class (IsVector a) => Graftable a where
    graft :: a -> a -> Vector (VectorScalar a) (VectorBasis a)

instance
    ( IsTree t, IsVector t
    -- [...]
    )
    => Graftable [t]
    where
    graft [] [] = vector []
    graft _ [] = vector Zero
    graft [] f2 = vector f2
    graft f [t] =
        linear ((: []) . buildTree (root t)) $ gl f $ children t
    graft f1 (t : f2) =
        linear perCoproductTerm $ deshuffleCoproduct f1
      where
        perCoproductTerm (f11, f12) = graft f11 [t] * graft f12 f2

gl  :: ( IsTree t, IsVector t
       -- [...]
       )
    => [t] -> [t] -> Vector (VectorScalar t) [t]
gl f1 f2 = linear perCoproductTerm $ deshuffleCoproduct f1
  where
    perCoproductTerm (f11, f12) = vector f11 * graft f12 f2
```

### 3.2.4 Substitution

We consider a substitution operation $\triangleright_d$ that substitutes the vertices decorated by $d$ of a forest by trees and attaches the branches of the respective vertices to the vertices of the corresponding trees.

**Example 3.13.** An example of substitution is:



in which we substitute the vertices decorated by 4 in by the trees of in all

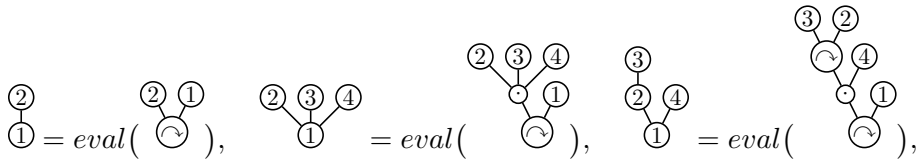
possible ways.

We implement substitution by recalling that the decorated vertices together with grafting and concatenation generate the whole space of decorated forests. That is, we have,

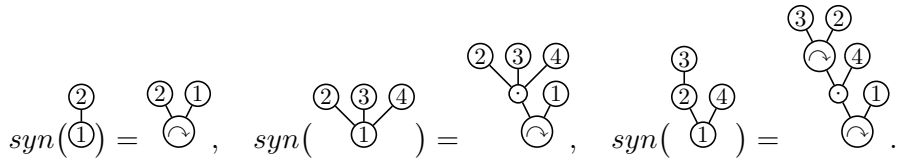
$$\tau = \pi \curvearrowright r, \quad \pi = \tau_1 \cdots \tau_n.$$

This implies that any forest can be represented as a syntactic tree with leaves being vertices and internal nodes being the operations of grafting or concatenation. Let the set of syntactic trees be denoted by $ST$, then, there is a map $eval : ST \to F$ which evaluates the syntacitc tree and $syn : F \to ST$ that deconstructs a forest into a syntacitc tree.

**Example 3.14.** Some examples of syntactic trees are:



or, analogously,



By translating the substitution operation to syntactic trees, we can significantly simplify its implementation. In the case of syntactic trees, the vertices of a forest $\pi$ become the leaves of the syntactic tree $syn(\pi)$, and the substitution $\triangleright_d$ reduces to simply replacing the leaves decorated by $d$. A detailed discussion of the syntactic tree implementation can be found in Section 4.

To implement the map `syn`, we need to define grafting and concatenation operations as decorations of internal vertices of a syntactic tree. To achieve this, we wrap these operations in `Operation a` objects.

**Implementation 3.15.** Wrapping of the grafting and concatenation operations:

```
graftOp :: (IsVector a, Graftable a) => Operation a
graftOp = Op
    -- [...]
    $ \ops ->
        case ops of
            [x, y] -> graft x y
            _ -> error "graftOp: arity"

concatOp :: (IsVector a, Monoid a) => Operation a
```

```
concatOp = Op
    -- [...]
    $ vector . mconcat
```

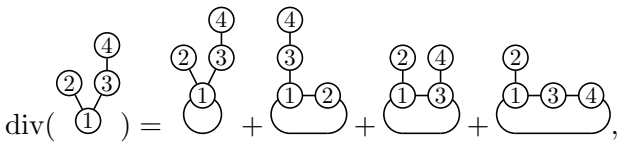
We use `graftOp` and `concatOp` to define the `syn` function, which is implemented as part of the `HasSyntacticTree` type class. We note that the implementation of the `eval` function and the `HasSyntacticTree` type class is discussed in Section 4.

**Implementation 3.16.** Implementation of the instance of the `HasSyntacticTree` type class for forests:

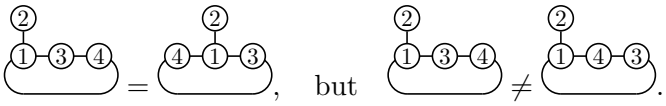
```
instance
    ( IsVector t
    , IsTree t
    -- [...]
    ) => HasSyntacticTree [t]
    where
    syn [t] = case children t of
        [] -> Leaf [t]
        _ -> Node graftOp
                [syn (children t), Leaf [buildTree (root t) []]]
    syn ts = Node concatOp $ map (syn . (: [])) ts
```

### 3.3 Decorated aromatic forests

Decorated aromatic forests arise in the field of geometric numerical integration, particularly when analyzing the divergence of vector fields represented by decorated rooted trees. Taking the divergence of such a vector field translates into taking the divergence of the decorated rooted trees, which involves connecting the root of the tree to each of its vertices in all possible configurations. For example,




where the cycles are oriented counterclockwise. These cycles are referred to as *aromas* and are defined as ordered forests that are invariant under cyclic permutations of their trees. This means that the aroma remains unchanged if the last tree in the cycle is moved to the first position, with all other trees shifting from position $i$ to $i+1$. For example,



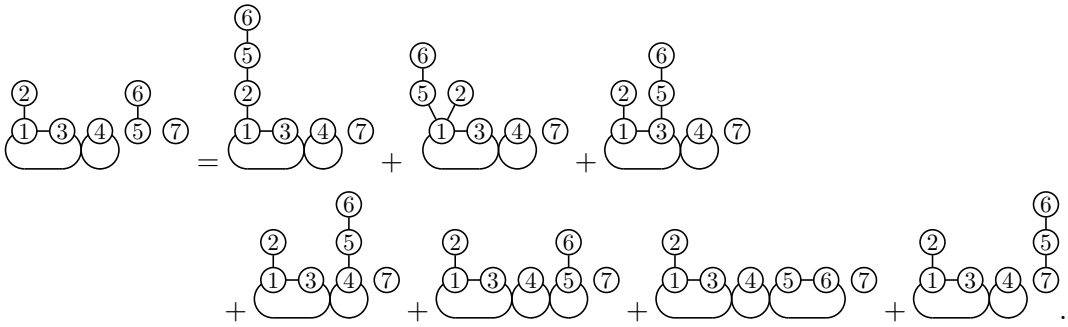
We define $\mathcal{A}$ as the vector space spanned by monomials of aromas (also referred to as *multi-aromas*), making it a free algebra of aromas, as discussed in Section 2.3. We then define aromatic forests as the tensor product $\mathcal{AF} := \mathcal{A} \otimes \mathcal{F}$. Below, we present the implementation of the planar version of aromatic forests, building on our discussion in Section 3.2.2.

**Implementation 3.17.** Planar implementation of the aromatic forests:

```
type PlanarAromatic t =
    ( [Cycle t]
    , [t]
    )
```

where `Cycle t` is a list of trees `[t]` invariant under cyclic permutations.

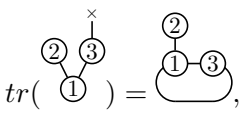
We implement an extended version of divergence, defined over planar aromatic forests, in which the leftmost root is connected to all vertices in all possible ways. For example,



This extension of divergence is studied in detail in [3]. To spare the reader from unnecessary technical details involved in handling aromas, we omit the implementation details of the `divergence` function.

### 3.3.1 Grafting

We refer to [2] for a detailed study of the algebraic structure of the grafting algebra of aromatic trees. Aromas are obtained as the image of the trace function $tr$, which takes a rooted tree with a marked vertex and replaces the marked vertex with a connection to the root. For example,



where $\times$ denotes the marked vertex. Following this, we extend the grafting operation to aromatic forests as defined below.

**Definition 3.18.** The **grafting** of two aromatic forests $(\omega_1, \pi_1)$ and $(\omega_2, \pi_2)$ with $\omega_1, \omega_2 \in \mathcal{A}$ and $\pi_1, \pi_2 \in \mathcal{F}$ is defined as follows:

$$(\omega_1, \pi_1) \curvearrowright (\omega_2, \pi_2) := \sum_{(\pi_1)} (\omega_1 \cdot (\pi_1^{(1)} \curvearrowright \omega_2), \pi_1^{(2)} \curvearrowright \pi_2),$$

where the grafting of $\pi \in \mathcal{F}$ onto a multi-aroma $\omega \in \mathcal{A}$ with $\omega = \omega_1 \cdot \omega_2$ is defined as

$$\pi \curvearrowright \omega = \sum_{(\pi)} (\pi^{(1)} \curvearrowright \omega_1) \cdot (\pi^{(2)} \curvearrowright \omega_2),$$

with grafting of $\pi \in \mathcal{F}$ onto an aroma $\omega = tr(\eta)$ with $\eta \in \mathcal{F}$ defined as

$$\pi \curvearrowright tr(\eta) = tr(\pi \curvearrowright \eta).$$

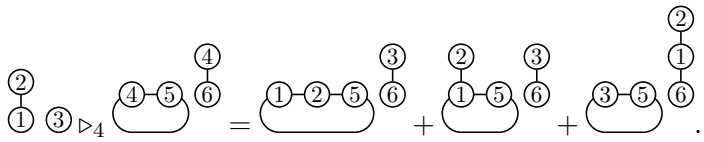
**Implementation 3.19.** Implementation of grafting of aromatic forests:

```
graftOnMultiAroma
    :: ( IsDecorated t, IsTree t, IsVector t
       -- [...]
       )
    => [t] -> [Cycle t] -> Vector (VectorScalar t) [Cycle t]
graftOnMultiAroma [] ma = vector (1 *^ ma)
graftOnMultiAroma _ [] = vector Zero
graftOnMultiAroma f [a] =
    linear ((1 *^) . (: []) . Cycle) $ (f `graft`) $ unCycle a
graftOnMultiAroma f (a : ma) =
    linear perCoproductTerm $ deshuffleCoproduct f
  where
    perCoproductTerm (x, y) =
        (x `graftOnMultiAroma` [a]) * (y `graftOnMultiAroma` ma)

instance
    ( IsDecorated t, IsTree t, IsVector t
    -- [...]
    )
    => Graftable (PlanarAromatic t)
    where
    graft (ma1, f1) (ma2, f2) =
        vector (ma1, [])
            * linear perCoproductTerm (deshuffleCoproduct f1)
      where
        perCoproductTerm (x, y) =
            linear (,[]) (x `graftOnMultiAroma` ma2)
```

```
                    * linear ([],) (y `graft` f2)
```

### 3.3.2 Substitution

The substitution operation on aromatic forests is defined analogously to the substitution on decorated forests in Section 3.2.4. For example,



Following [2], we note that an aromatic forest can be decomposed into a syntactic tree using three operations: grafting, concatenation, and trace. Therefore, we implement the function `syn` using `graftOp` and `concatOp` introduced in Section 3.2.4, along with a new operation `traceOp`. We omit the technical details of the implementation of `syn` and `traceOp` for brevity.

# 4 Syntactic trees

As mentioned in Section 3.2.4, representing objects as syntactic trees can simplify the implementation of operations on the objects of interest. In our case, we use syntactic trees to streamline the implementation of the substitution of (aromatic) forests; however, this approach can also be applied to implement morphisms of algebras, provided there is a systematic way to represent the objects of the algebra using a set of products and generators. In this section, we discuss the implementation of the syntactic tree data structure, the construction and evaluation of syntactic trees, and the operations performed on them.

We note that this approach is similar to that of the theory of operads, which uses trees to represent the composition of operations.

**Definition 4.1.** A **syntactic tree** is a tree whose leaves are decorated by elements of a set of generators and whose internal vertices are decorated by operations.

To allow operations to be used as decorations of internal vertices, we define the `Operation a` data type, which contains the name of the operation, its TeX representation, the arity of the operation (the number of arguments it takes), and the function that evaluates the operation on a list of elements of type `a`.

**Implementation 4.2.** We define the `Operation a` and `SyntacticTree a` data types:

```
data Operation a = Op
    { name :: String
    , tex :: String
```

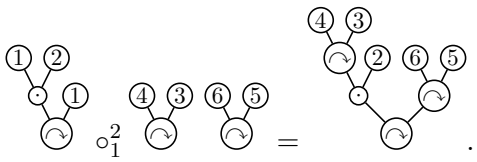
```
    , arity :: Int
    , func :: [a] -> Vector (VectorScalar a) (VectorBasis a)
    }

data SyntacticTree a
    = Node (Operation a) [SyntacticTree a]
    | Leaf a
```

All objects that can be represented using a syntactic tree must be instances of the `HasSyntacticTree` type class, which contains the function `syn`. The `syn` function constructs a syntactic tree from an object, while the `eval` function evaluates a syntactic tree to obtain the object it represents. We note that the `eval` function is already available and does not need to be defined separately for each algebra.

**Implementation 4.3.** The type class `HasSyntacticTree` wich contains `syn` and the function `eval` are implemented as follows:

```
class (IsVector a) => HasSyntacticTree a where
    syn :: a -> SyntacticTree a

eval
    :: ( IsVector a
       -- [...]
       )
    => SyntacticTree a -> Vector (VectorScalar a) a
eval (Leaf a) = vector a
eval (Node op as)
    = linear (func op) $ product $ map (linear (: []) . eval) as
```

Let the space of syntactic tree $ST$ be endowed with a composition $\circ_d^n : ST \times ST^n \to ST$ that replaces the $i$-th leaf decorated by $d$ of the first argument by the $i$-th syntactic tree of the second argument. For example,



We omit the implementation of the `compose` function for the sake of brevity. Its type signature is provided below. We note that the implementation is more general than the original definition. Instead of replacing only the leaves, it can also replace syntactic subtrees. Additionally, it returns `Nothing` if the number of subtrees to be replaced differs from the number of syntactic trees in the second argument.

**Implementation 4.4.** Type signature of the composition operation:

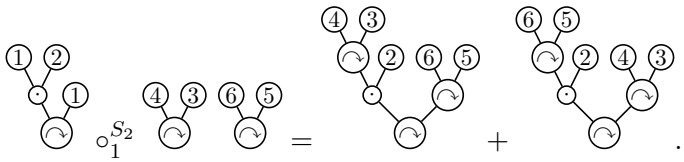
```
compose :: SyntacticTree a -> [SyntacticTree a] -> SyntacticTree a
           -> Maybe (SyntacticTree a)
```

where the first argument is the syntactic tree to be replaced, the second argument is the list of syntactic trees to replace with, and the third argument is the syntactic tree to replace in.

We go further and define symmetric composition $\circ_d^{S_n} : ST \times ST^n \to ST$ to be the composition $\circ_d^n$ which is summed over all permutations of the right operand. That is,

$$\gamma \circ_d^{S_n} (\gamma_1 \cdots \gamma_n) = \sum_{\sigma \in S_n} \gamma \circ_d^n (\gamma_{\sigma(1)} \cdots \gamma_{\sigma(n)}).$$

For example,



**Implementation 4.5.** Implementation of the symmetric composition:

```
symmetricCompose
    :: ( IsVector a
       -- [...]
       )
    => SyntacticTree a -> [SyntacticTree a] -> SyntacticTree a
       -> Vector (VectorScalar (SyntacticTree a)) (SyntacticTree a)
symmetricCompose x ops obj =
    mconcat
        $ map
            ( \perm_ops -> case compose x perm_ops obj of
                Just g -> vector (1 *^ g)
                Nothing -> vector Zero
            )
        $ permutations ops
```

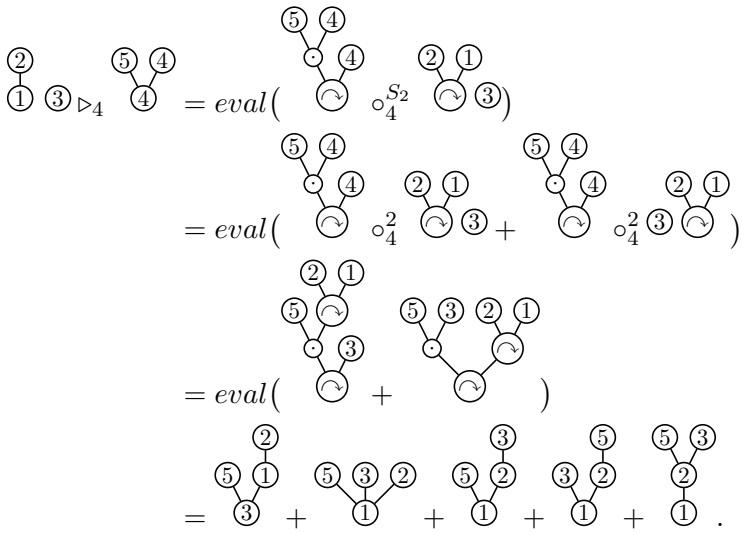
We note that once we represent the (aromatic) forests from Sections 3.2.4 and 3.3.2 as syntactic trees, the substitution operation corresponds to the symmetric composition.

**Definition 4.6. Substitution** operation $\triangleright_d$ is the operation on (aromatic) forests that corresponds to the symmetric composition on syntactic trees, that is,

$$(\tau_1 \cdots \tau_n) \triangleright_d \pi_2 = eval\big(syn(\pi_2) \circ_d^{S_n} (syn(\tau_1) \cdots syn(\tau_n))\big),$$

if $\pi_2$ has exactly $n$ vertices decorated by $d$. Otherwise, $(\tau_1 \cdots \tau_n) \triangleright_d \pi_2 = 0$.

**Example 4.7.** Let us compute the substitution seen in Example 3.13 step by step:



**Implementation 4.8.** Implementation of the substitution operation:

```
substitute
    :: ( HasSyntacticTree a
       , IsVector a
       -- [...]
       )
    => a -> [a] -> a -> Vector (VectorScalar a) a
substitute x gens obj =
    linear eval $ symmetricCompose (syn x) (map syn gens) $ syn obj
```

## References


[1] P. Chartier, E. Hairer, and G. Vilmart. Algebraic structures of B-series. *Foundations of Computational Mathematics*, 10:407–427, 2010.

[2] G. Fløystad, D. Manchon, and H. Z. Munthe-Kaas. The universal pre-Lie–Rinehart algebras of aromatic trees. In *Geometric and Harmonic Analysis on Homogeneous Spaces and Applications*, pages 137–159. Springer International Publishing, 2021.

[3] A. Laurent, R. I. McLachlan, H. Z. Munthe-Kaas, and O. Verdier. The aromatic bicomplex for the description of divergence-free aromatic forms and volume-preserving integrators. *Forum of Mathematics, Sigma*, 11:e69, 2023.